

Lab 3: Architecture Justification

Faculty Research Grant & Publication Tracker · SRN: PES1UG24CS437

Architecture Selection: “We chose Layered Architecture for the Faculty Research Grant & Publication Tracker System.”

Architectural Style Analysis

Style	Fit for this system	Verdict
Layered	Clean split of UI, grant/publication rules and data; one database keeps balances and approvals consistent; simple to build and test for a single deanery portal.	Selected
Microservices	Independent scaling is unnecessary for ~3 user roles in one university; splitting grants, expenses and audit across services forces distributed transactions for the overdraft check.	Rejected
Client-Server	Centralised data is good, but on its own it does not separate business rules from the UI, so rules such as overdraft blocking risk being duplicated or bypassed.	Partly (deployment)

Two Reasons for the Choice

1. **Grant money must stay consistent.** FR-001 requires that no expense is recorded if it would overdraw a grant unless the Dean approves it. In the layered design the Grant Fund Manager (business layer) checks the remaining balance and inserts the expense in one database transaction through the Data Access component, and balances are derived from approved expenses rather than stored separately. With microservices the balance, expense and audit data would live in different services, so two simultaneous expenses could both pass the check.
2. **The requirements map directly onto layers.** The portal has three roles (Faculty, Co-Author, Dean) and five functional areas, all used inside one research deanery. Each area becomes one business-layer component (Grant Fund Manager, Publication Manager, Burn-Up Analytics) behind a single Web Portal UI and a shared data layer. This mirrors the Epic 1 code (pages → ledger.ts → schema.sql), so later epics plug into the business layer without changing the UI or database, and without the deployment and monitoring overhead of many services.

Security Advantage

Single enforcement point for every rule. The layers are closed: the UI can only reach the business layer through its interfaces (IGrantFunds, IPublications, IBurnUp), and only the Data Access component talks to the database, using parameterised queries. Role checks such as “only the Dean may approve an overdraft” live in the business layer, so they cannot be skipped by calling a page or server action directly. Every financial approval and status change also goes through IAuditLog to the append-only, hash-chained Audit Ledger, which makes tampering detectable (NFR-001).

Performance Benefit

In-process calls and queries close to the data. Calls between layers are ordinary function calls inside one application, with no network hops or serialisation between services. Burn-Up Analytics computes cumulative spend with a single aggregate SQL query over one database instead of joining data from several services, which keeps the dashboard within the 5-second target (NFR-002). The slow, sometimes-unavailable external citation index is isolated behind the Citation Index Adapter with a timeout and manual fallback, so it never delays grant or dashboard pages.

Trade-off Accepted

Layered systems scale as one unit and add a little call overhead per layer. This is acceptable because load is limited to one university's faculty; if a part ever needs to scale on its own, its clean interface (for example ICitationLookup) lets it be extracted into a separate service later.